

Сверточные нейронные сети. Автокодировщики.

ЛЕКЦИЯ №5



Темы

1. Сверточные нейронные сети
2. Автоэнкодеры
3. Архитектура автокодировщика
4. Примеры

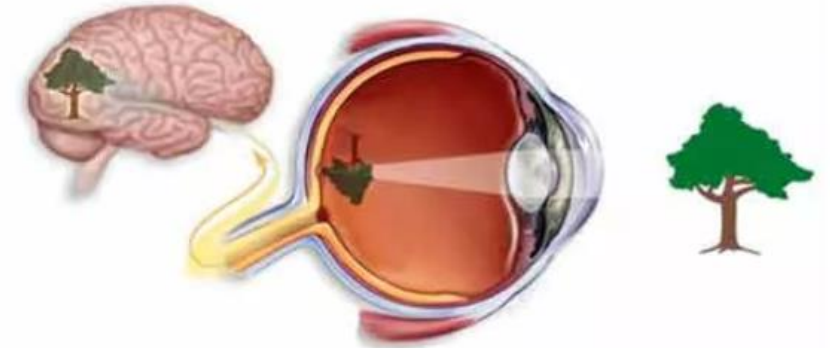
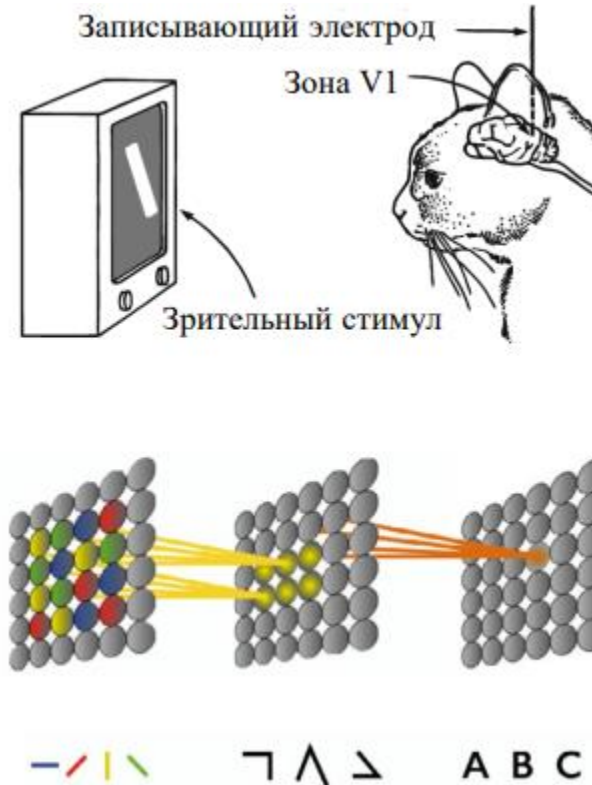
Темы

1. Сверточные нейронные сети

История

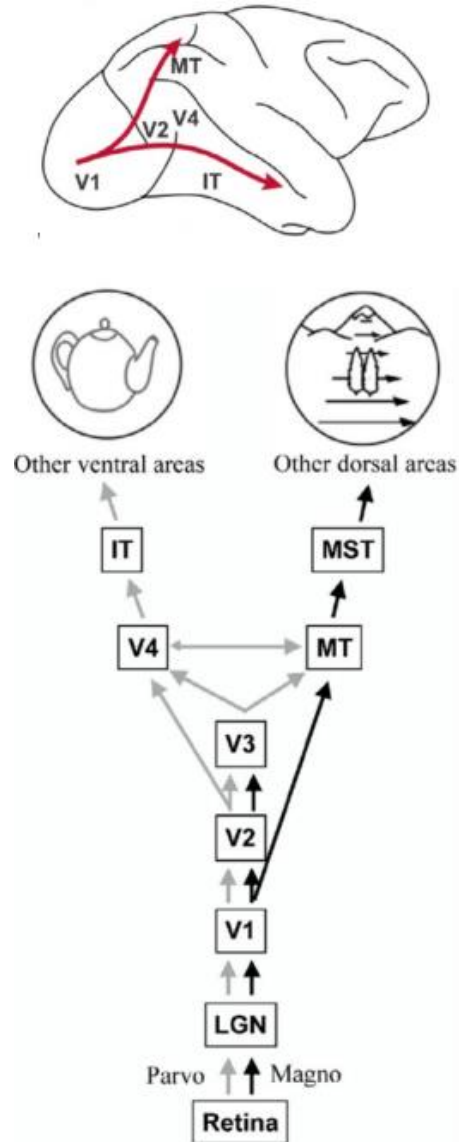
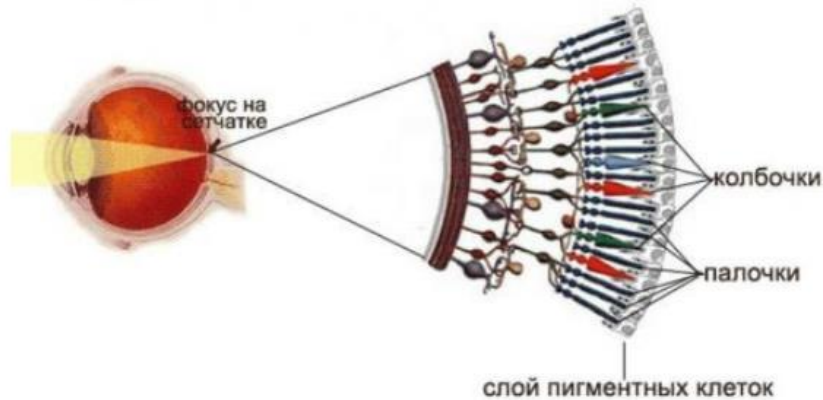


Давид Хубель и Торстен Визель
Нобелевская премия 1981 г.



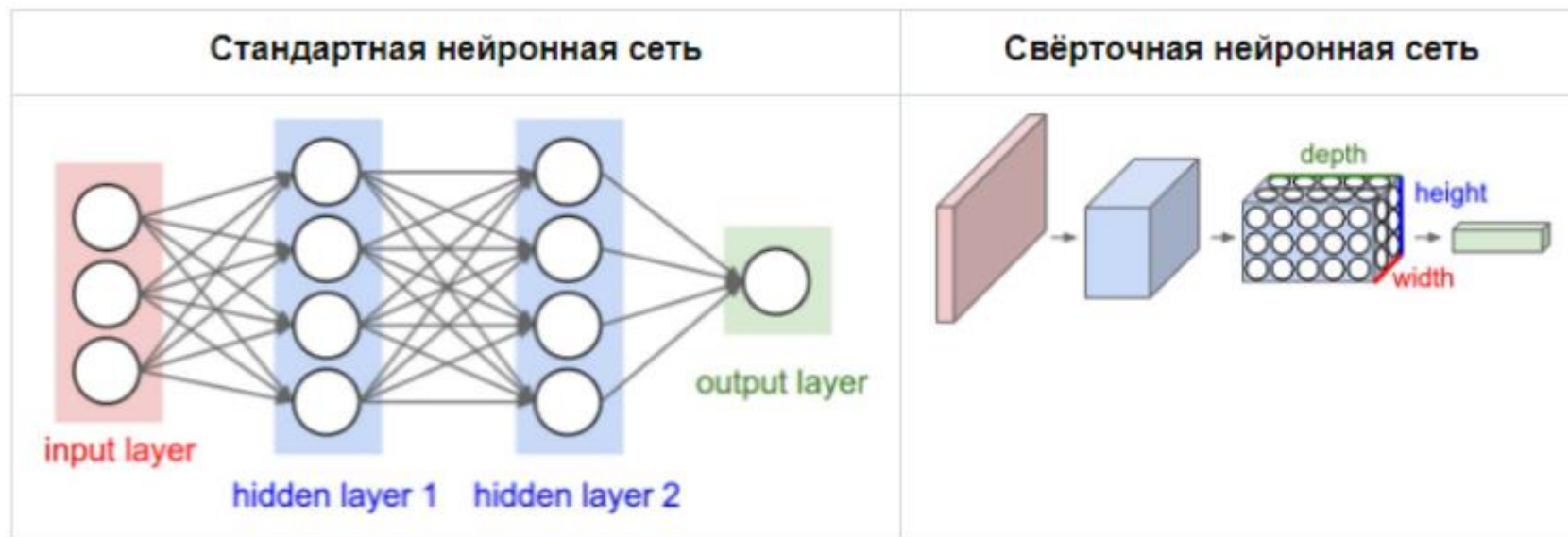
История

▼ "Входы" и "Выходы"

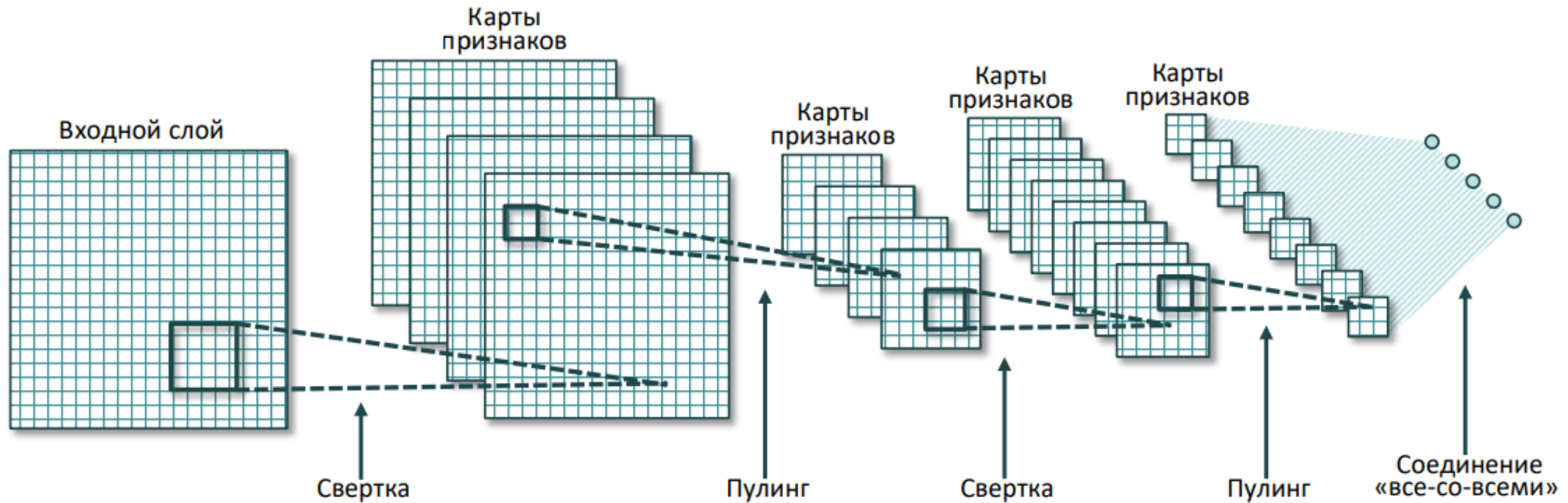


- Изображение формируется светом, который стимулирует сетчатку (retina), генерирующую электрические импульсы
- Нейроны сетчатки передают эти импульсы по зрительному нерву в зону V1, расположенную на затылке
- Зона V1 организована в виде двумерной плоской карты, повторяющей структуру сетчатки
- Определенные нейроны зоны V1 генерируют сигнал для зоны V2, если «видят» линию в соответствующей области
- Нейроны зоны V2 распознают углы и генерируют соответствующие сигналы для зоны V3, и так далее, вплоть до «бабушкиного нейрона»
- «Бабушкин нейрон» генерирует сигнал, если в поле зрения попадает родная бабушка

СНС



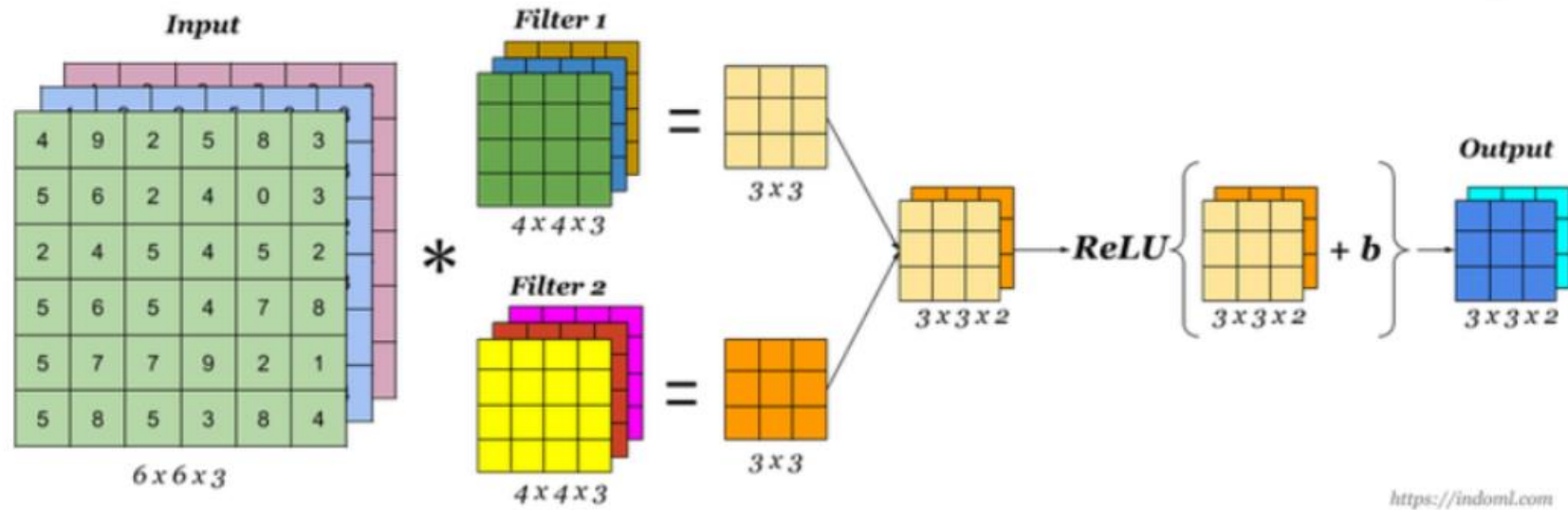
Концепция СНС



Входной слой логически представляется как матрица пикселей

- Каждый внутренний слой логически представляется как совокупность карт признаков (feature map) [карт активации (activation map)], имеющих одинаковые размеры
- Выходной слой имеет обычную линейную структуру и связывается с последним внутренним слоем по принципу «все-со-всеми»
- Нейроны входного слоя и всех внутренних слоев связываются избирательно (не все со всеми)
- Связи строятся двумя чередующимися способами: 1) свертка, 2) пулинг
- Свертка увеличивает количество карт признаков, сохраняя их размер (при использовании padding), либо незначительно уменьшая
- Пулинг уменьшает размеры карт признаков с сохранением их количества

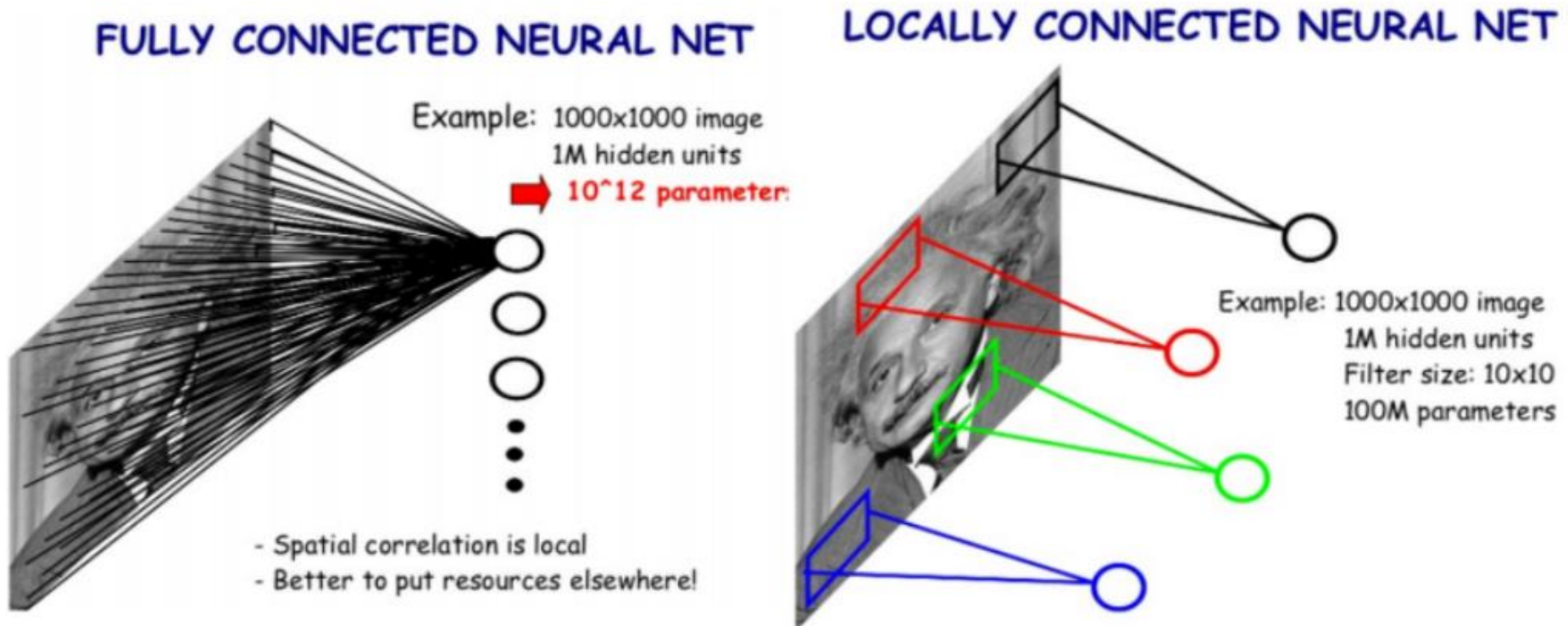
Сверточный слой Conv2D



```
.add(Conv2D(32, (3, 3), padding='same', activation='relu', strides=(1,1)))
```

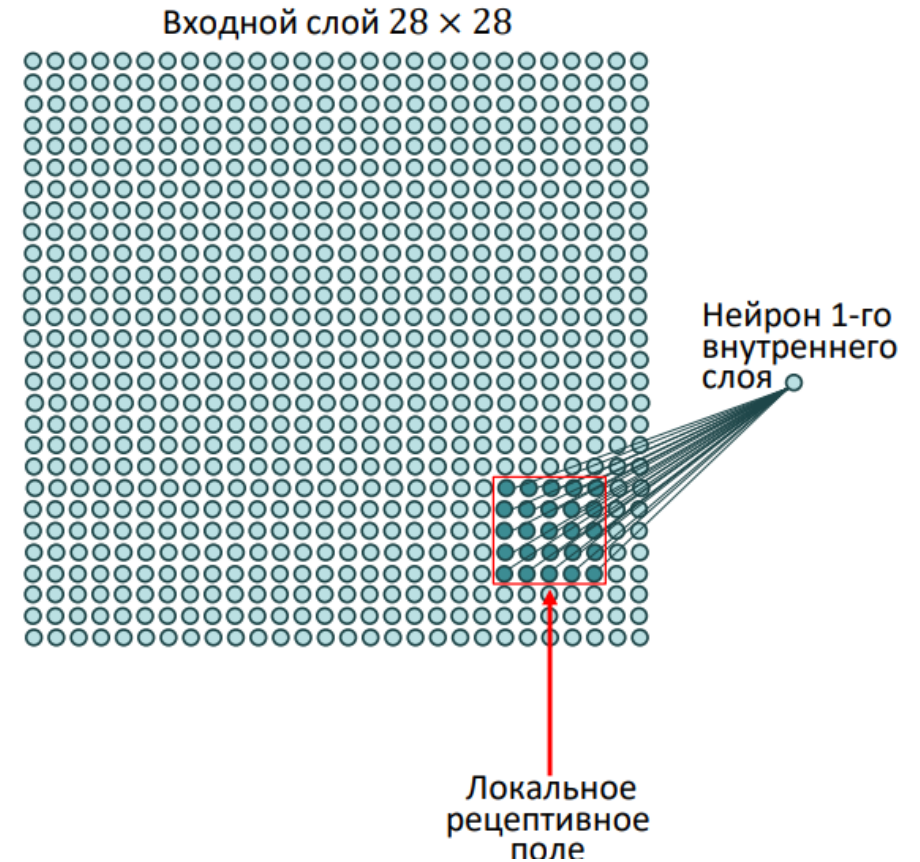
1. Первый параметр **32** - количество ядер (фильтров) свертки.
2. Второй параметр **(3, 3)** – размер ядра свертки.
3. Третий параметр **padding='same'** - тип заполнения краев, нужен для сохранения размеров изображения, по умолчанию значение **padding='valid'**.
4. Четвертый параметр **activation='relu'** – указание функции активации.

Сверточный слой Conv2D

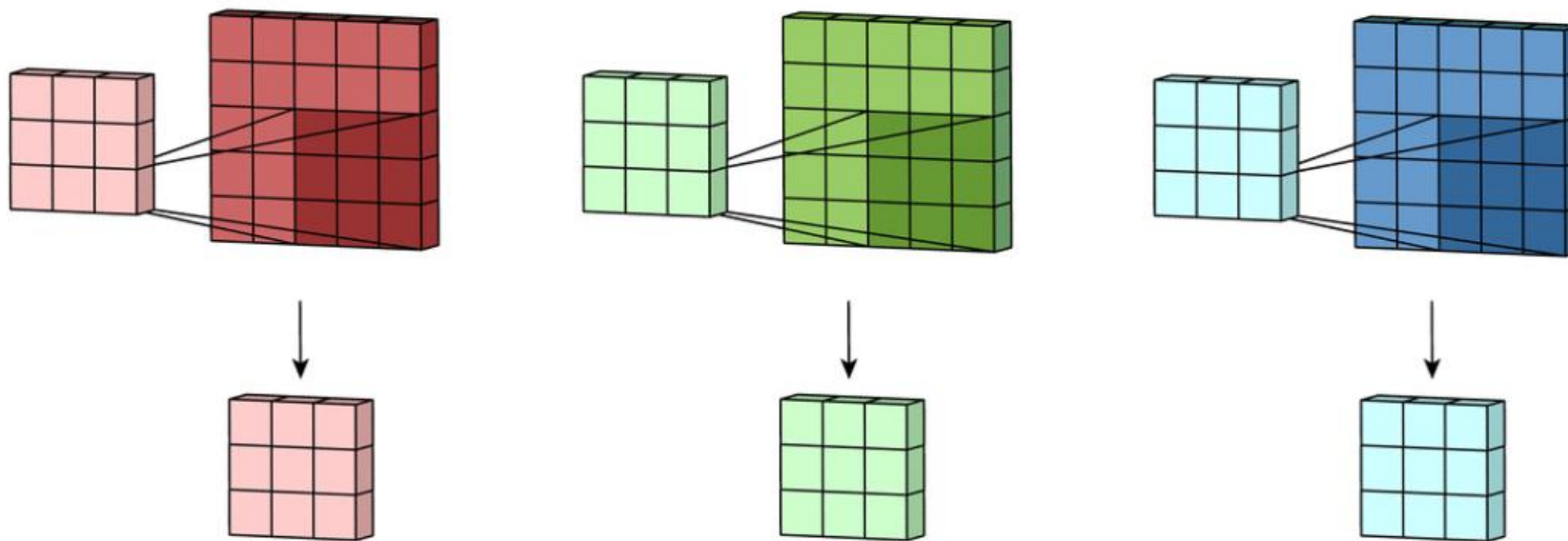


Концепция СНС

- **Карта признаков** (карта активации)- обработанное ядром свертки исходное изображение
- **Ядро свертки** – тензоры одного размера (задаются при настройке). Количество ядер в слое определяет глубину выходного массива.
- **Свертка** – операция вычисления нового значения на основе выбранного пикселя и ядра свертки.
- Каждый нейрон первого внутреннего слоя соединяется с группой входных нейронов, образующих прямоугольный слайдер размером $n \times n$, называемый *локальным рецептивным полем*
- Слайдер локального рецептивного поля устанавливается в левый верхний угол матрицы входного слоя и проходит по матрице нейронов до правого нижнего угла



Принцип работы Conv2D



Алгоритм свёртки:

1. фильтр накладывается на левую верхнюю часть входящего изображения;
2. производится поэлементное умножение значений фильтра и значений пикселей изображения;
3. полученные значения складываются, сумма будет результатом свертки области наложения (одно число);
4. фильтр перемещается дальше по изображению (за смещение отвечает параметр `strides`, по умолчанию равен `(1,1)`, т.е. смещение на 1 пиксель вправо, при достижении конца строки сдвиг вниз на 1 пиксель, и вновь с начала строки);
5. в новом положении окна фильтра производится поэлементное умножение значений фильтра ...
6. повтор до тех пор, пока аналогичным образом не будут обработаны все участки.

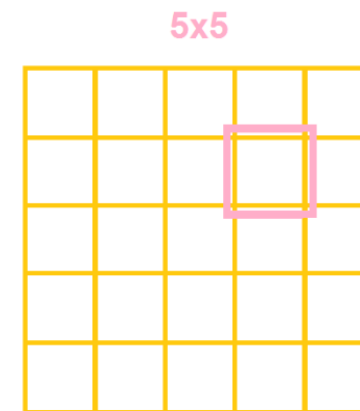
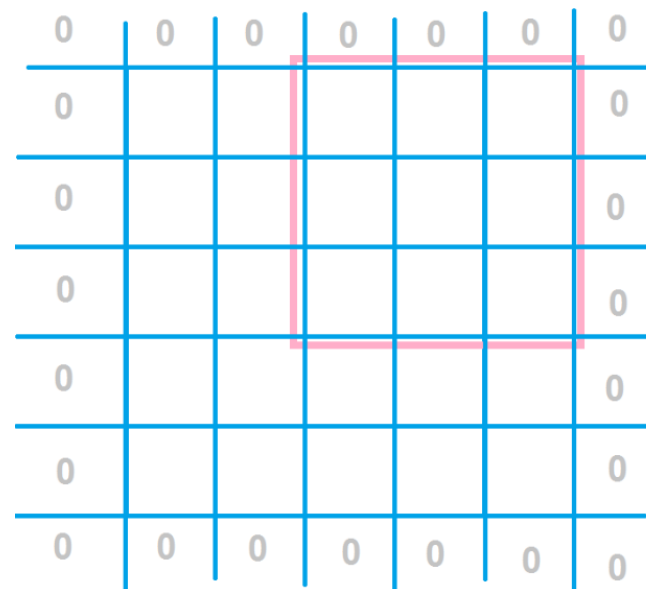
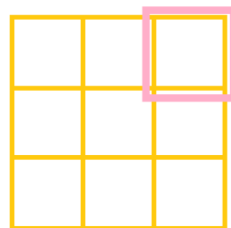
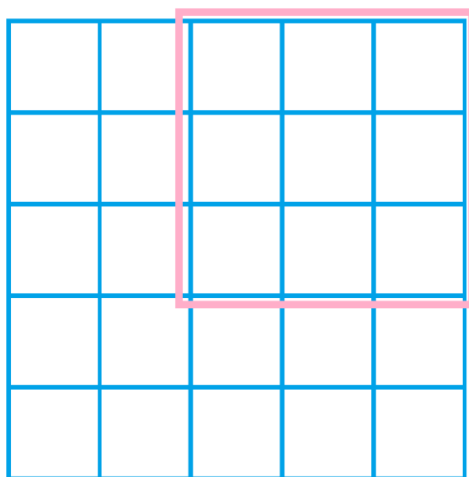
Принцип работы Conv2D

Заполнение (Padding)

Малый размер ядра фильтра - потеря в размерах выходного слоя невелика. Если же это значение увеличивается, то и размеры выходного слоя уменьшается по отношению к оригинальному размеру входного изображения.

Чтобы избежать потерь в разрешении выходных изображений, используют дополнительный параметр – заполнение (padding), позволяющий расширить полученное изображение по краям до того же размера, что и на входе свертки.

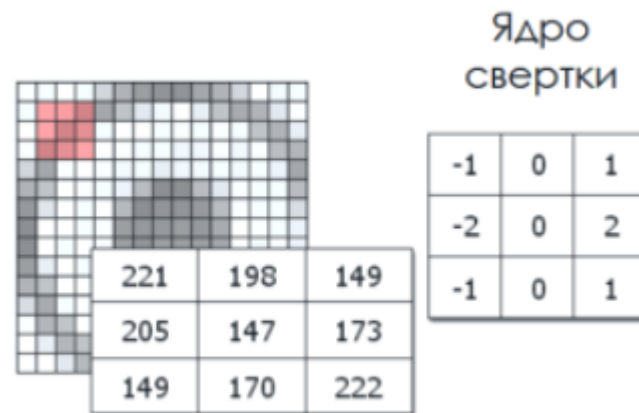
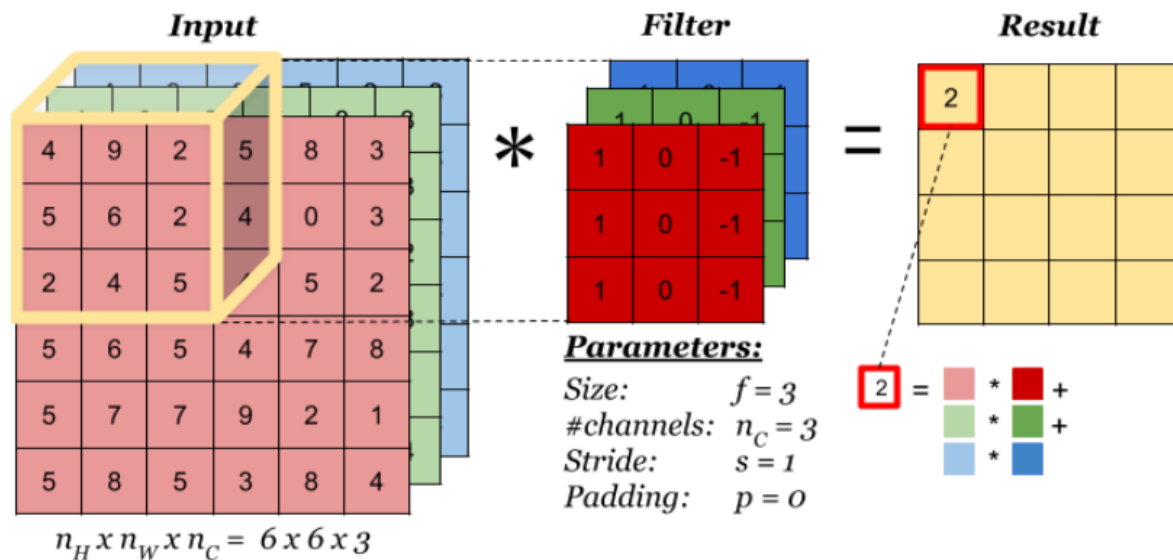
Пример: если мы по картинке размером 5 x 5 пикселей проходим ядром 3 x 3, то в конце получим потерю в размерах.



Решение: добавляем параметр `padding='same'`, тогда Keras дополнит выходное изображение по внешней границе нулями.

Принцип работы Conv2D

Размеры ядра – длина, ширина, глубина



$$N(x,y) = 221 * (-1) + 198 * 0 + 149 * 1 + 205 * (-2) + 147 * 0 + 173 * 2 + 149 * (-1) + 170 * 0 + 222 * 1 = -63$$

Принцип работы Conv2D



Без изменений ▾

0	0	0
0	1	0
0	0	0

Применить



Выделение границ по X ▾

-1	-2	-1
0	0	0
1	2	1

Применить



Выделение границ по Y ▾

-1	0	1
-2	0	2
-1	0	1

Применить



Принцип работы Conv2D



Повышение резкости ▾

-1	-4	-1
-4	26	-4
-1	-4	-1

Применить





Размытие Гаусса ▾

1	2	1
2	4	2
1	2	1

Применить



Принцип работы Conv2D

Алгоритм свёртки:

1. Фильтр накладывается на левую верхнюю часть входящего изображения;
2. Производится поэлементное умножение значений фильтра и значений пикселей изображения;
3. Полученные значения складываются, сумма будет результатом свертки области наложения (одно число);
4. Фильтр перемещается дальше по изображению (за смещение отвечает параметр `strides`, по умолчанию равен $(1,1)$, т.е. смещение на 1 пиксель вправо, при достижении конца строки сдвиг вниз на 1 пиксель, и вновь с начала строки);
5. В новом положении окна фильтра производится поэлементное умножение значений фильтра ...
6. Повтор до тех пор, пока аналогичным образом не будут обработаны все участки.

Принцип работы Conv2D

Слой подвыборки (Pooling-слой)

Основные задачи слоев типа Pooling:

1. Распознавание объектов вне зависимости от масштаба;
2. Факт наличия признака важнее знания места его точного положения на изображении.

Слой уменьшает размер изображения, выбирая максимальное (MaxPooling), среднее (AveragePooling) или суммарное (SumPooling) значение из окна фильтра..

Feature Map

6	6	6	6
4	5	5	4
2	4	4	2
2	4	4	2

Max Pooling

6	6
4	4

Average Pooling

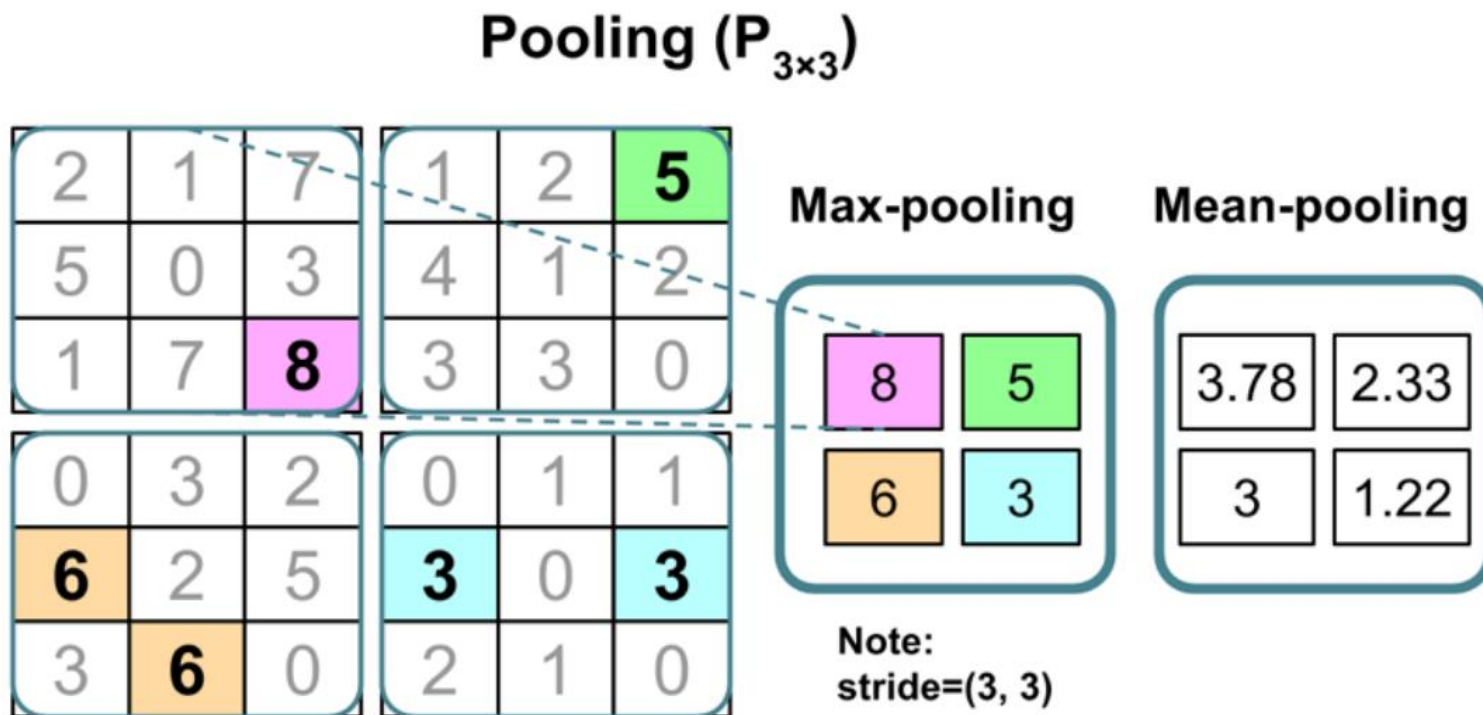
5.25	5.25
3	3

Sum Pooling

21	21
12	12

Принцип работы Conv2D

Слой Pooling: нет тренируемых параметров



Принцип работы Conv2D

Финальная классификация данных

После прохождения через сверточные слои и слои подвыборки данные необходимо систематизировать и классифицировать. Для этого применяют полносвязные слои **Dense**,

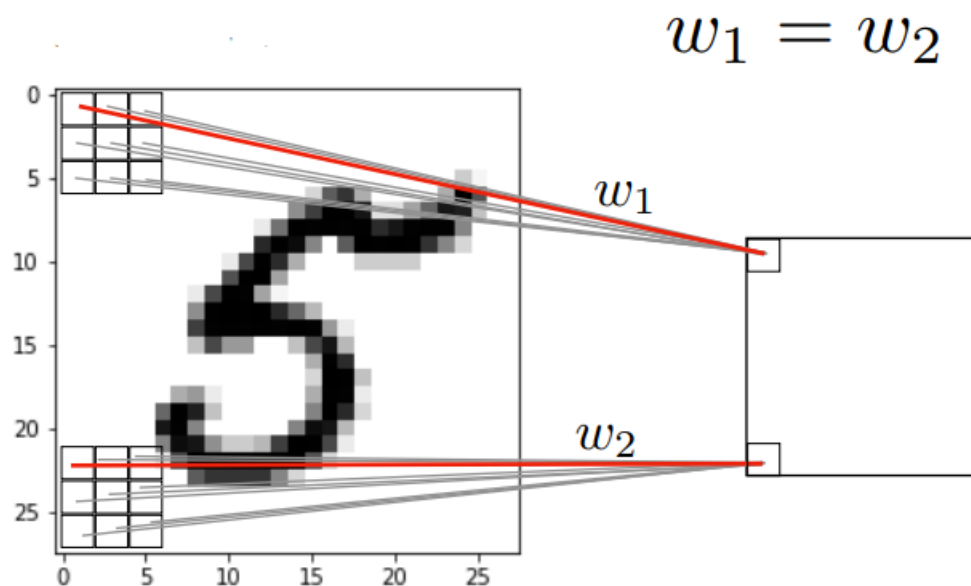
1. Flatten() - переводит многомерные входные данные в одномерный вектор, при этом размеры данных по всем осям перемножаются; дополнительных параметров нет. Например, входящее изображение формы **(28, 28, 3)** преобразуется в вектор формы **(2352)**.
2. Reshape(...) - смена формы данных.

1	1	0
4	2	1
0	2	1



1
1
0
4
2
1
0
2
1

Backpropagation CNN



Optional averaging

$$w_1 := w_2 := w_1 - \eta \cdot \frac{1}{2} \left(\frac{\partial \mathcal{L}}{\partial w_1} + \frac{\partial \mathcal{L}}{\partial w_2} \right)$$

Принцип работы Conv2D

Преимущества

1. Один из лучших методов по распознаванию и классификации изображений
2. Меньшее количество настраиваемых весов по сравнению с полносвязной нейронной сетью
3. Удобное распараллеливание вычислений (карты признаков обрабатываются независимо)
4. Относительная устойчивость к повороту и сдвигу распознаваемого изображения
5. Обучение при помощи классического метода обратного распространения ошибки

Потенциальные проблемы

1. Много варьируемых параметров сети:
 - количество слоев;
 - размерность ядра свертки;
 - шаг сдвига слайдера;
 - степень уменьшения размера при пулинге;
 - функция для пулинга;
 - вид функции активации и др.

Conv2D

```
Conv2D(  
    filters,                # количество фильтров (выходных каналов)  
    kernel_size,            # размер матриц ядер  
  
    strides=(1, 1),         # шаг свёртки  
    padding="valid",        # расширение матрицы изображения  
    data_format=None,       # положение оси канала в форме изображения  
    dilation_rate=(1, 1),   # разрежение свёртки  
    groups=1,               # количество параллельных групп свёрток  
    activation=None,        # функция активации  
    use_bias=True,          # наличие bias  
    kernel_initializer="glorot_uniform", # инициализатор весов ядра  
    bias_initializer="zeros", # инициализатор весов bias  
    kernel_regularizer=None, # регуляризация ядра  
    bias_regularizer=None,   # регуляризация bias  
    activity_regularizer=None, # регуляризация выхода слоя  
    kernel_constraint=None,  # ограничения весов ядра  
    bias_constraint=None,    # ограничения весов bias  
    **kwargs,  
)
```

Слой `tf.keras.layers.Conv2D` создает ядра свёртки, которые сворачиваются с входным тензором для создания тензора выходных данных. Если `use_bias` имеет значение `True`, создается вектор смещения, который добавляется к выходным данным после свёртки. Если функция активации не равна `None`, она применяется к выходу. При использовании `Conv2D` слоя в качестве первого в модели указывается аргумент ключевого слова `input_shape` (кортеж целых чисел или `None`), например `input_shape = (128, 128, 3)` для цветных изображений 128x128 RGB в `data_format = "channels_last"`.

Conv2D

Форма входных данных

4D-тензор формы batch_shape + (channels, rows, cols), если data_format='channels_first'

Форма выходных данных

4D-тензор формы batch_shape + (filters, new_rows, new_cols), если data_format='channels_first'

Аргументы

filters

Тип: int.

Описание: размер канального выходного пространства (количество каналов).

Примеры:

- filters = 3 – на выходе 3 канала
- filters = 32 – на выходе 32 канала.

kernel_size

Тип: целое число int или (int, int).

Описание: Указание высоты и ширины окна двумерной свёртки. Может быть одним целым числом, чтобы указать одно и то же значение для всех пространственных измерений. Размер указывается сразу для всех ядер свёртки слоя.

Примеры:

- kernel_size = 3 – ядро свёртки 3x3;
- kernel_size = (3, 3) – ядро свёртки 3x3;
- kernel_size = (2, 5) – ядро свёртки 2x5.

Ключевые слова

input_shape

Тип: tuple или list.

Описание: задание формы входного 4D-тензора (без задания размера батча).

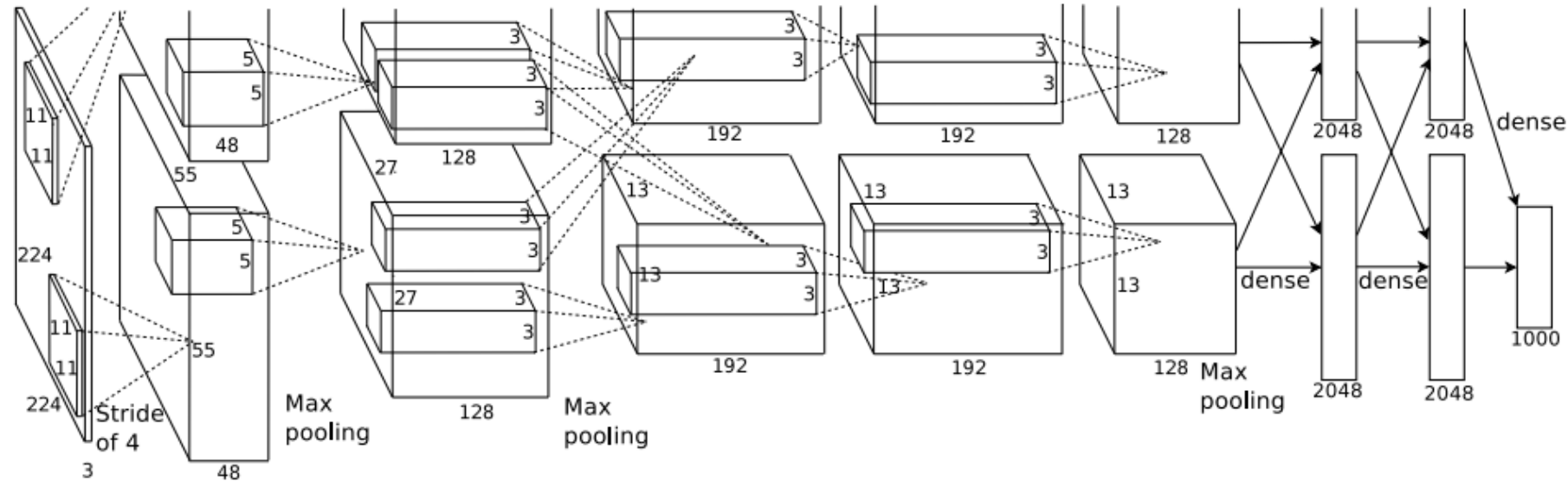
Примеры:

- input_shape = (224, 224, 3) – изображение размера 224x224 с 3-мя каналами ("цветное");
- input_shape = (28, 28, 1) – изображение размера 28x28 с 1-м каналом ("чёрно-белое");
- input_shape = (2, 256, 256) – изображение размера 256x256 с 2-мя каналами (при параметре data_format=channels_first);
- input_shape = (None, None, 3) – изображение любой ширины и высоты с 3-мя каналами.

data_format

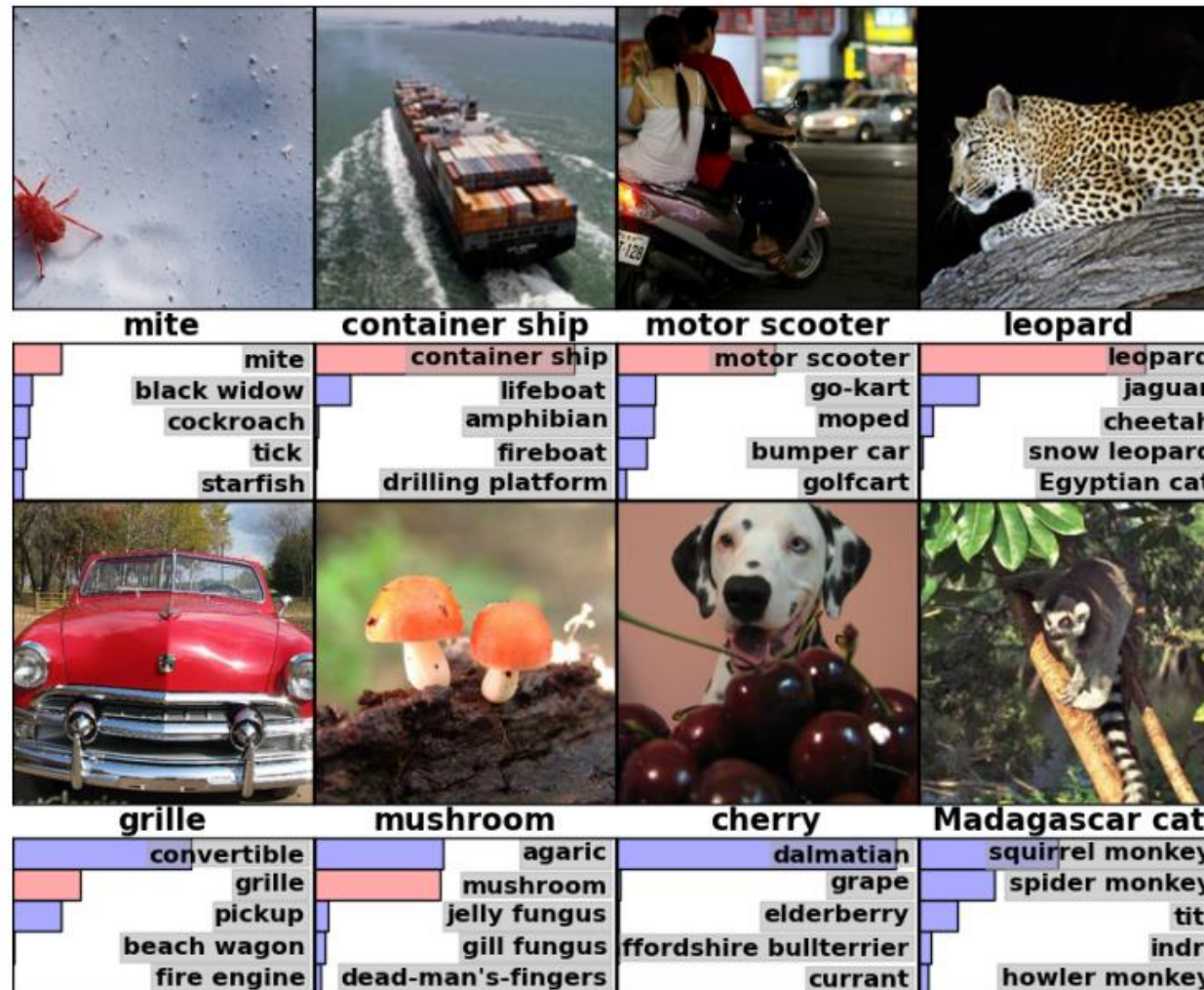
Значения: 'channels_last' (по умолчанию) или 'channels_first'.

AlexNet & ImageNet



An illustration of the architecture of our CNN, explicitly showing the delineation of responsibilities between the two GPUs. One GPU runs the layer-parts at the top of the figure while the other runs the layer-parts at the bottom. The GPUs communicate only at certain layers. The network's input is 150,528-dimensional, and the number of neurons in the network's remaining layers is given by 253,440–186,624–64,896–64,896–43,264–4096–4096–1000.

AlexNet & ImageNet

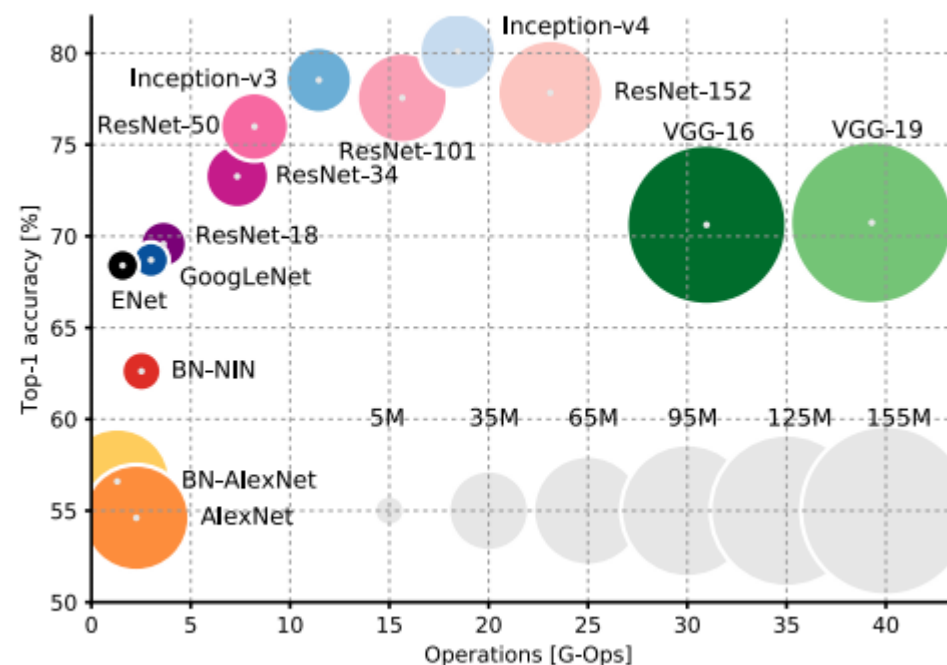
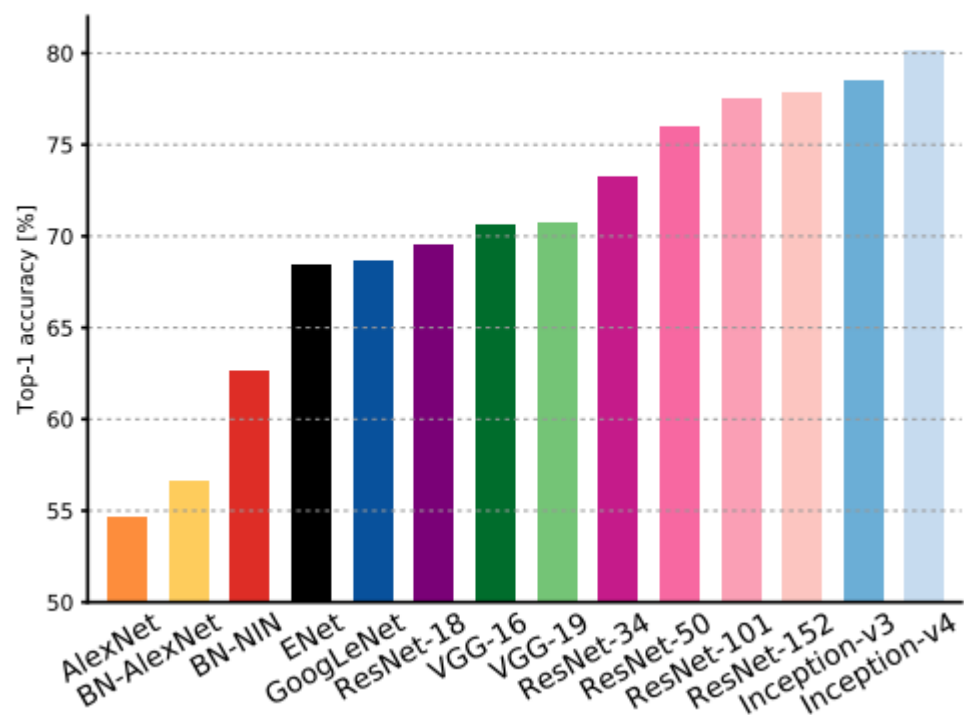


~1.2 млн
изображений и 1000 классов

Accuracy - top-5 Performance.

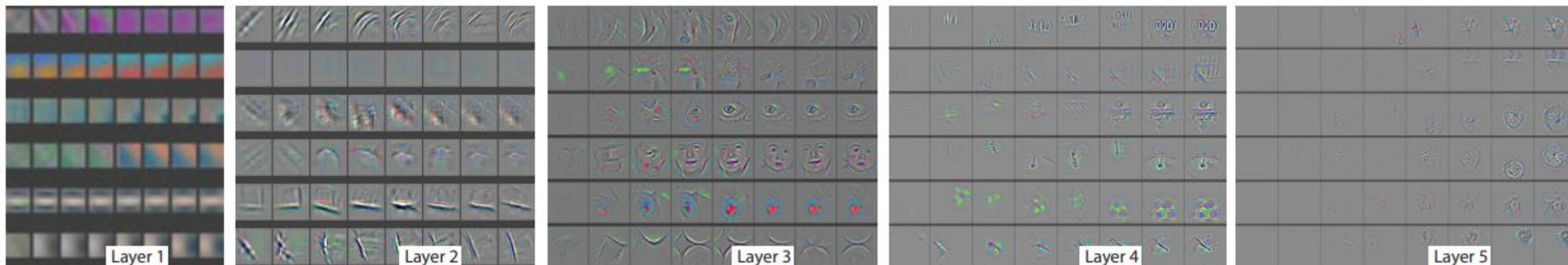
Correct prediction - true
label - one of the top 5

Архитектуры CNN



Canziani, A., Paszke, A., & Culurciello, E. (2016). An analysis of deep neural network models for practical applications. arXiv preprint arXiv:1605.07678.

Визуализация CNN

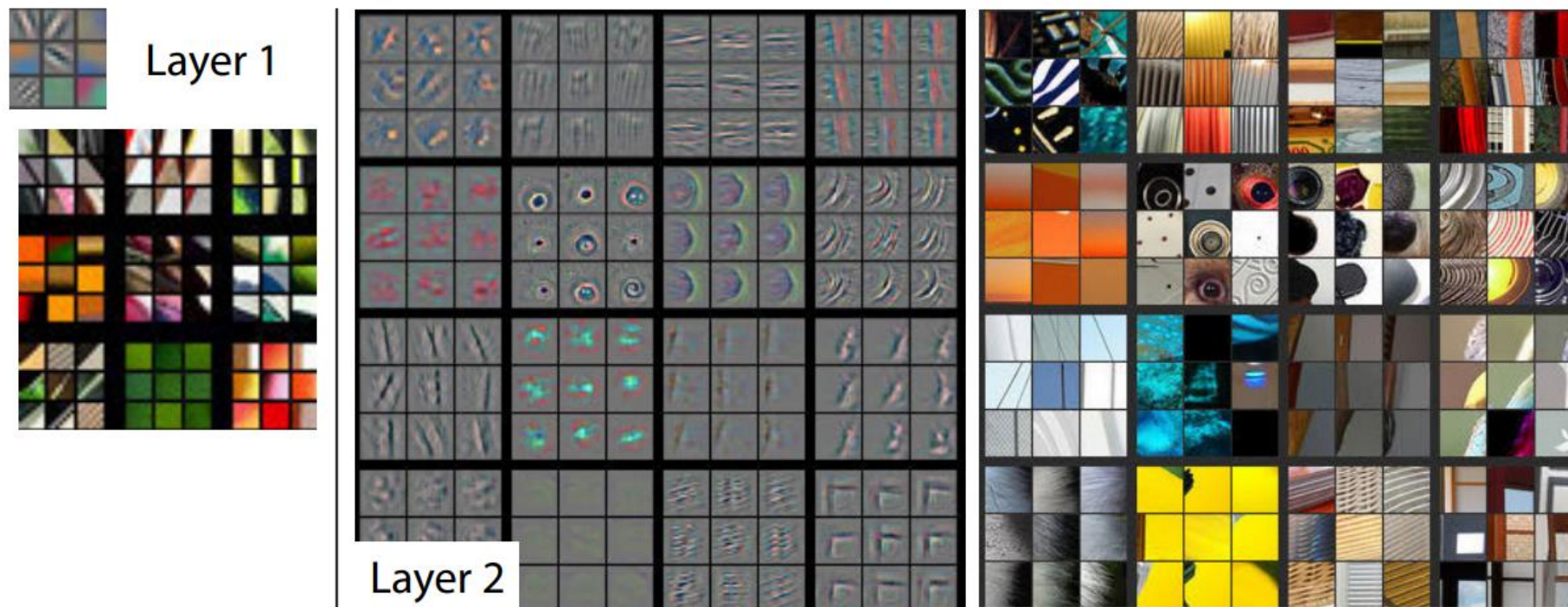


Характеристики полностью обученной модели

Для слоев 2-5 представлены преобразование в латентное пространство..

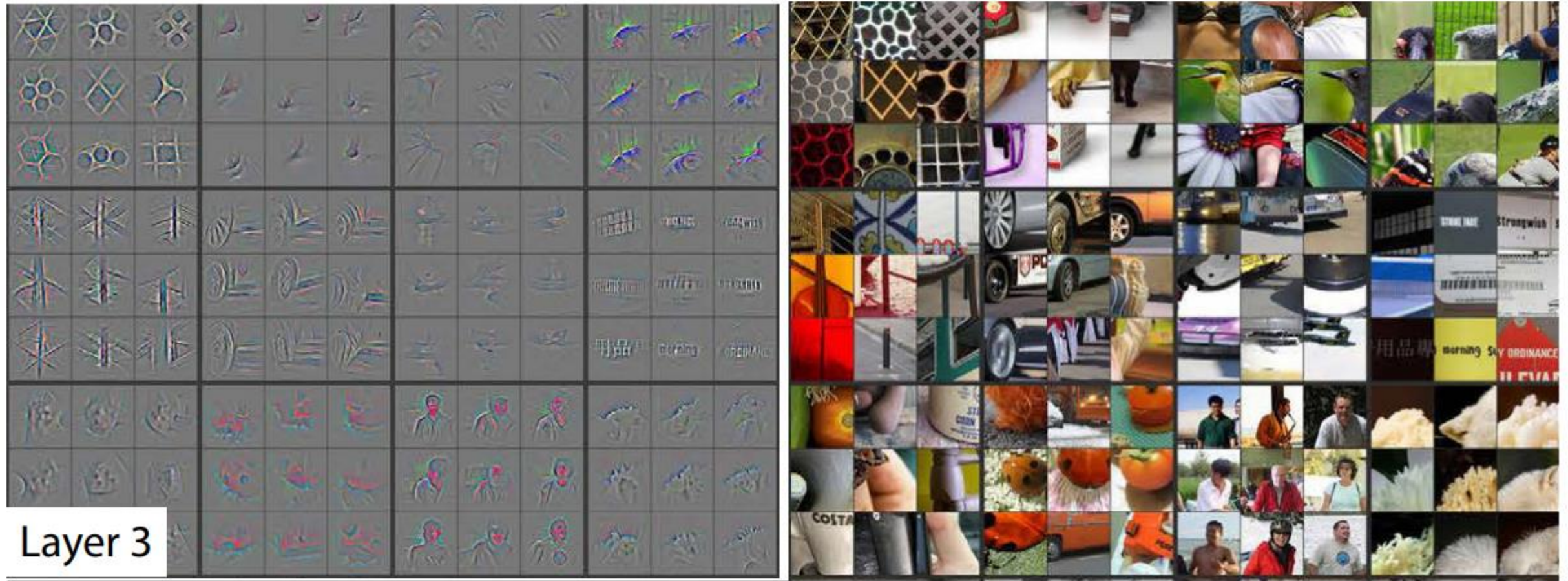
Представлены паттерны изображений для каждой карты признаков.

Визуализация CNN



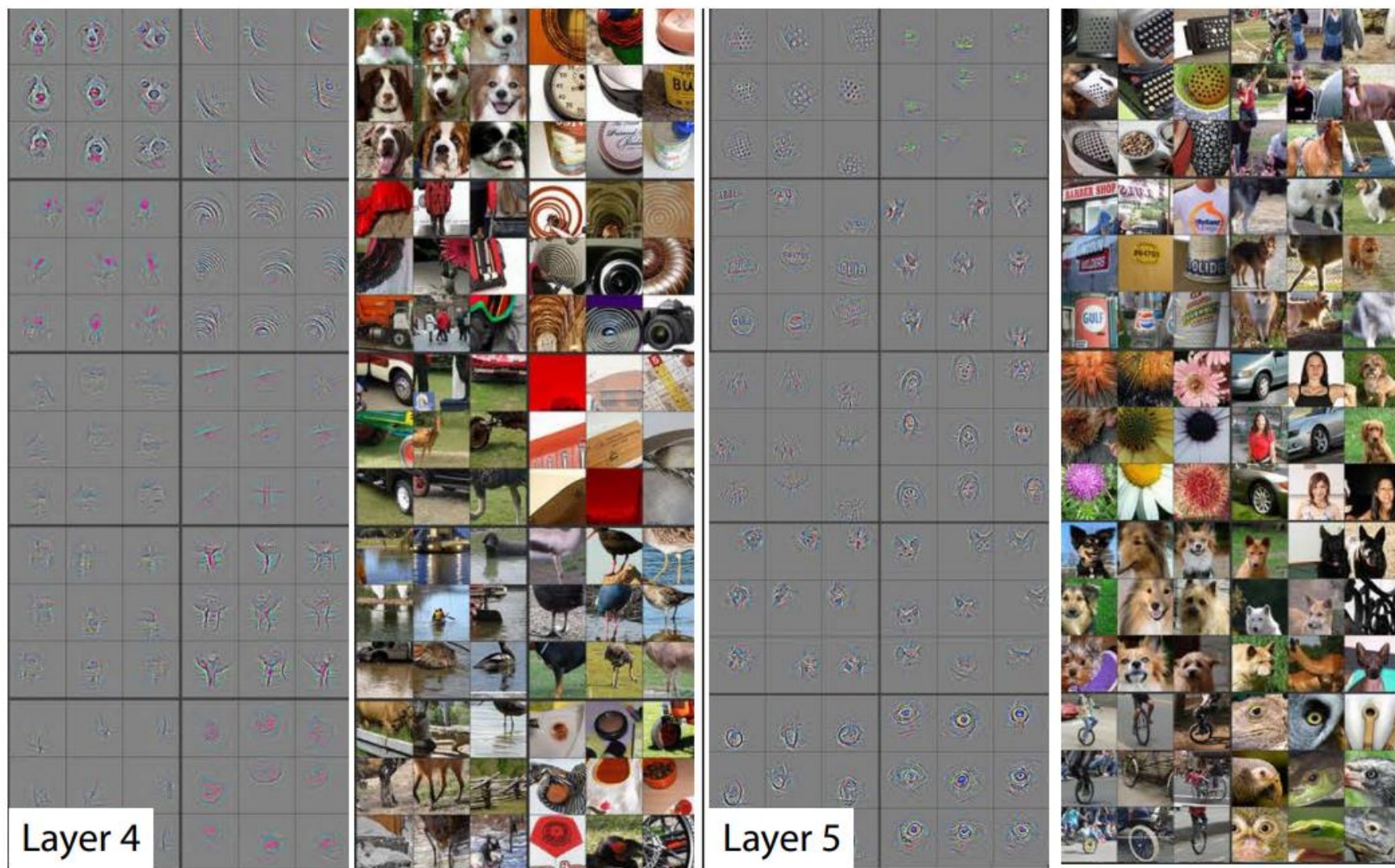
Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In European conference on computer vision (pp. 818-833). Springer, Cham

Визуализация CNN



Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In European conference on computer vision (pp. 818-833). Springer, Cham

Визуализация CNN



Zeiler, M. D., & Fergus, R. (2014, September). Visualizing and understanding convolutional networks. In European conference on computer vision (pp. 818-833). Springer, Cham

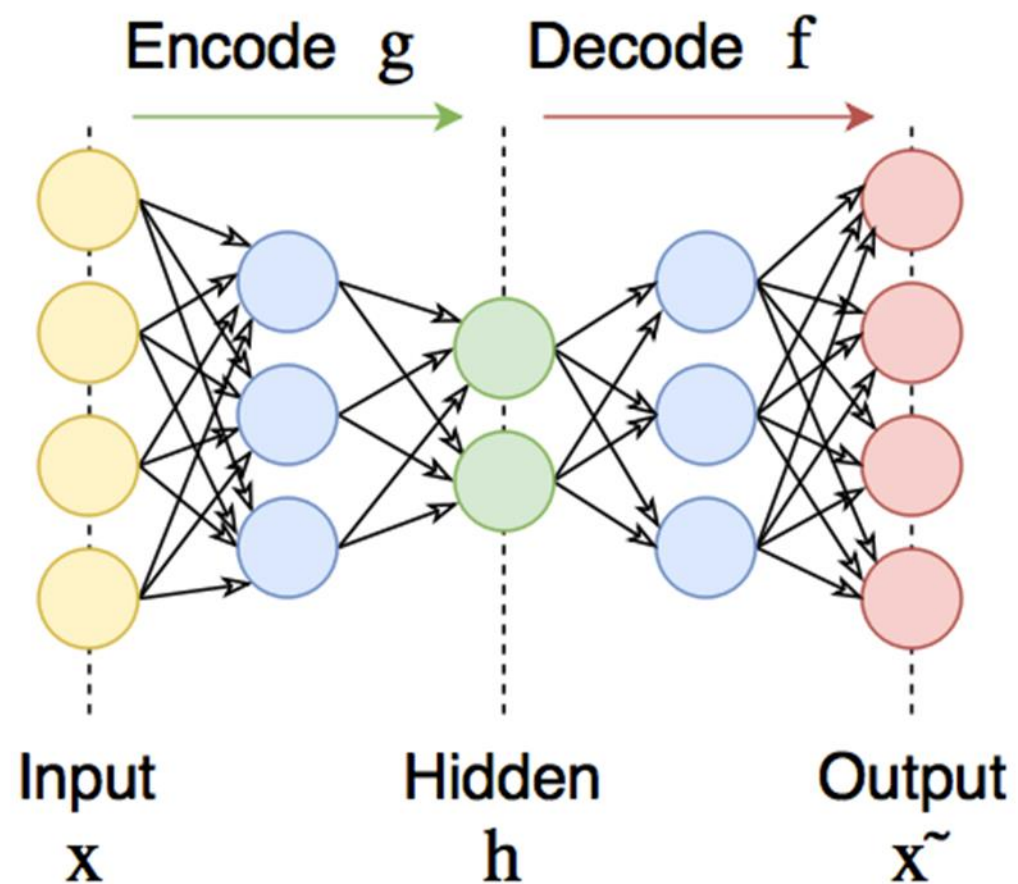
2. Автоэнкодеры

Автокодировщик

- Автоэнкодеры (AE)
- Вариационные автоэнкодеры (VAE)
- Вариационные автоэнкодеры с условием (CVAE)

Автокодировщик

- Автоэнкодеры (АЕ)



Автоэнкодеры (АЕ)

ТИПЫ АВТОЭНКОДЕРОВ:

- глубокие автоэнкодеры;
- сверточные автоэнкодеры;
- Denoising автоэнкодеры.

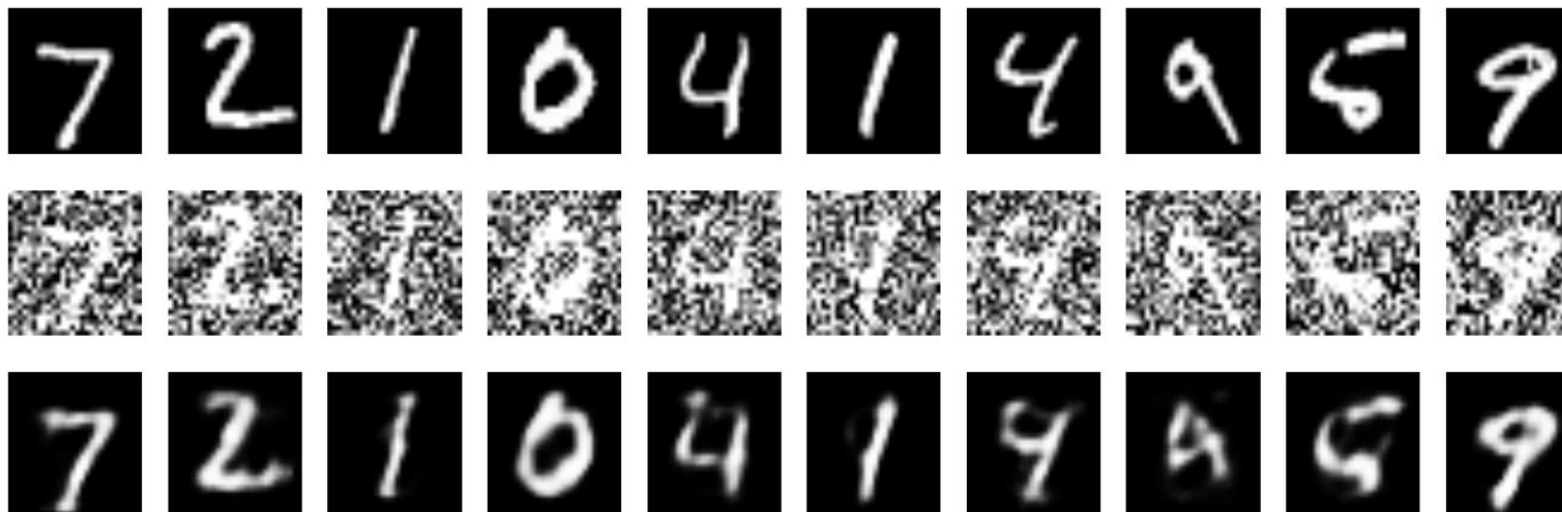
Применение:

- получение проекции многомерных данных;
- уменьшение размерности многомерных данных для визуализации;
- кодирование данных;
- обучение глубоких сетей (до 2014 г.).

Автоэнкодеры (АЕ)

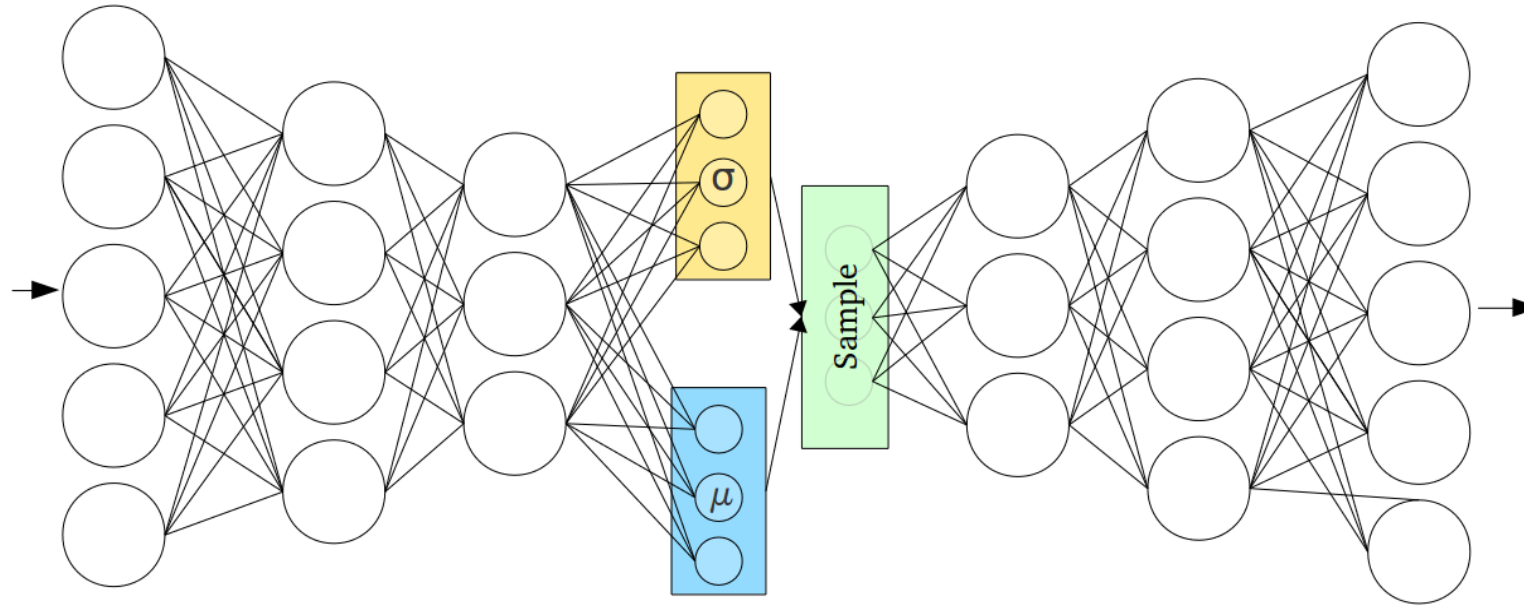
Denoising автоэнкодер – удаление шума

Скрытое пространство – не непрерывно



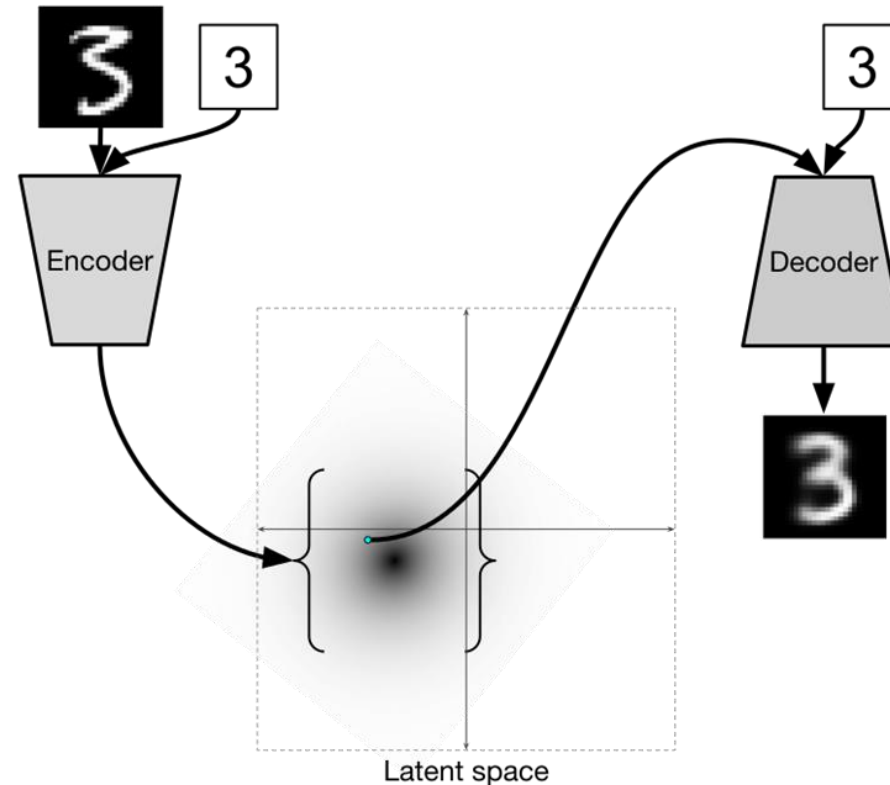
Вариационные Автоэнкодеры (VAE)

Вариационный автоэнкодер (Variational Autoencoder – VAE) — автокодировщик (генеративная модель, которая учится отображать объекты в заданное скрытое пространство и обратно), основанный на вариационном выводе.



Вариационные автоэнкодеры с условием (CVAE)

Вариационный автоэнкодер с условием — автокодировщик основанный на вариационном выводе, который накладывает условия на входное пространство.



3. Архитектура автокодировщика

Автокодировщик

Автокодировщик (англ. autoencoder) – специальная архитектура ИНС, позволяющая применять обучение без учителя при использовании метода обратного распространения ошибки.

Цель обучения автокодировщиков: найти внутреннюю структуру в данных.

Применение автокодировщиков:

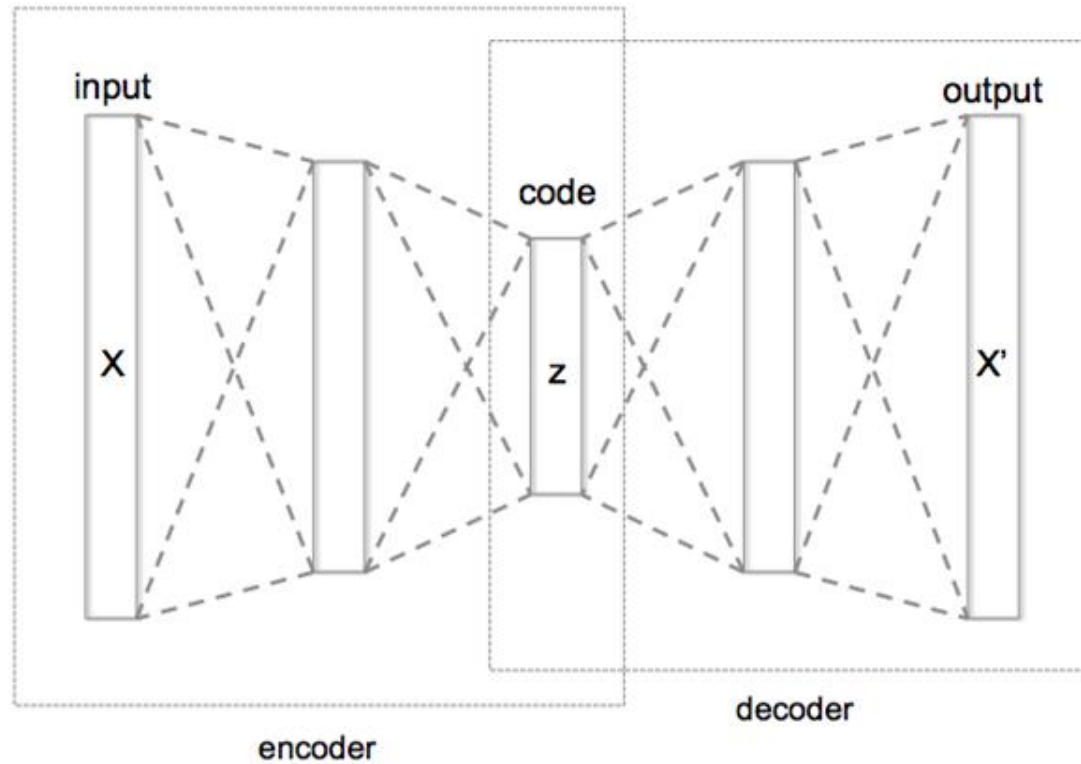
- улучшение качества изображения (увеличение размера, шумоподавление и т. д.),
- генерация новых данных по заданному образцу (например, раскрашивание черно-белых картинок в цветные),
- поиск выбросов.

Автокодировщик

Простейшая архитектура автокодировщика – сеть прямого распространения без обратных связей, наиболее схожая с персептроном и содержащая входной слой, промежуточный слой и выходной слой.

В отличие от персептрона выходной слой автокодировщика должен содержать столько же нейронов, сколько и входной слой.

Автокодировщик



Автокодировщик состоит из двух частей: энкодера (кодирует выборку X в свое внутреннее представление Z) X и декодера (восстанавливает исходную выборку) X' .

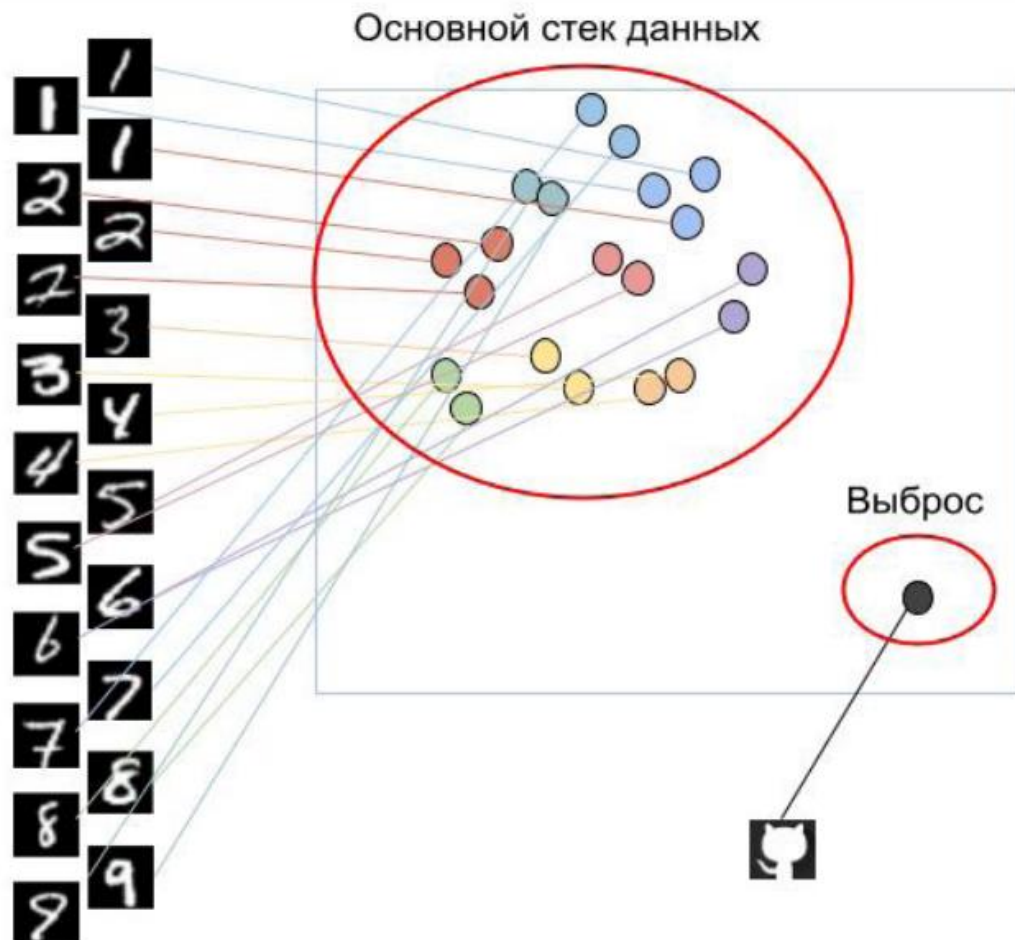
Автокодировщик



Рассмотрим пример работы автоэнкодера на восстановлении изображений из библиотеки MNIST.

Одно из основных предназначений таких автокодировщиков – снижение размерности исходного пространства.

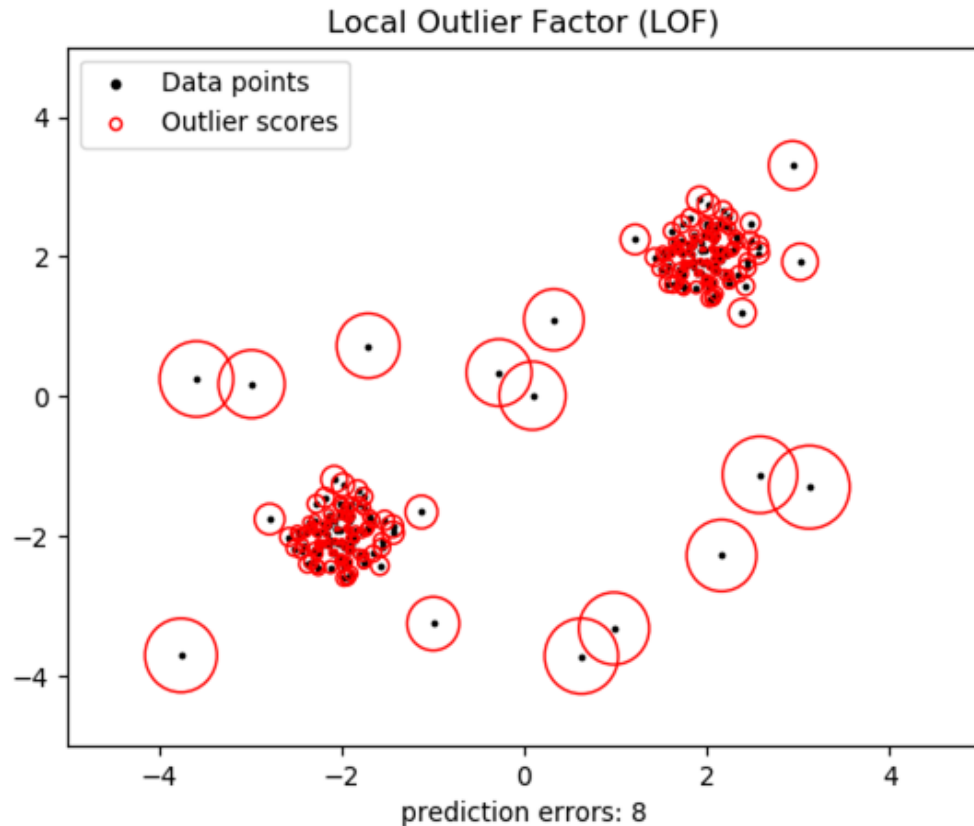
Поиск выбросов (outlier detection)



Выброс – объекты, которые встречаются очень редко (мошеннические транзакции и т.п.)
Для каждого экземпляра данных нужно определить, выброс это, или нет
Пропорции данных: один выброс на 1000, 10000 и больше объектов

Применяется при сильной разбалансировке базы

Поиск выбросов



Применение автокодировщика для поиска выбросов

- Обучаем автокодировщик только на обычных данных
- При распознавании оцениваем, насколько хорошо автокодировщик восстановил данные на выходе
- Автокодировщик не обучался на выбросах, поэтому будет плохо их восстанавливать
- Ошибка восстановления больше заданного порога – значит данный объект является выбросом

4. Примеры

Пример 1. Определение выбросов

Решение задачи определения выбросов среди изображений цифр и одежды, где цифры считаются нормальными изображениями, а изображения с одеждой – выбросами.

Архитектура автоэнкодера

Используются слои свертки **Conv2D** и слои снижения размерности **MaxPooling2D**.

Блоки в энкодере идентичны

Состав блока энклдера:

- Двумерная свертка.
- Нормализация.
- Двумерная свертка.
- Нормализация.
- Снижение размерности.

Каждый блок завершается слоем **MaxPooling2D**, который уменьшает размерность данных.

Архитектура декодера

2 блока декодера, представляющие собой обратные действия для энкодера.

Состав блока декодера:

- Повышение размерности.
- Нормализация.
- Двумерная свертка.
- Нормализация
- Двумерная свертка.
- Нормализация.

После двух блоков декодера идет финальный слой двумерной свертки декодера, который выдает итоговое изображение.

Пример

Архитектура автоэнкодера

Энкодер

```
def baseAutoencoder(shape=(112,80,1)): # зададим функцию создания базового автокодировщика
    img_input = Input(shape) # задаём входные размеры

    x = Conv2D(32, (3, 3), padding='same', activation='relu')(img_input) # входные данные передаем на слой двумерной свёртки
    x = BatchNormalization()(x)
    x = Conv2D(32, (3, 3), padding='same', activation='relu')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D()(x) # передаём на слой подвыборки, снижающий размерность поступивших на него данных

    x = Conv2D(64, (3, 3), padding='same', activation='relu')(x)
    x = BatchNormalization()(x)
    x = Conv2D(64, (3, 3), padding='same', activation='relu')(x)
    x = BatchNormalization()(x)
    z = MaxPooling2D()(x) # передаём на слой подвыборки
```

Пример

Архитектура автоэнкодера

Декодер

```
x = Conv2DTranspose(64, (2, 2), strides=(2, 2), padding='same', activation='relu')(z) # слой разжимает данные(с 28*28 на 56*40)
x = BatchNormalization()(x)
x = Conv2D(64, (3, 3), padding='same', activation='relu')(x)
x = BatchNormalization()(x)
x = Conv2D(64, (3, 3), padding='same', activation='relu')(x)
x = BatchNormalization()(x)

x = Conv2DTranspose(32, (2, 2), strides=(2, 2), padding='same', activation='relu')(x) # слой разжимает данные(с 56*40 на 112*80)
x = BatchNormalization()(x)
x = Conv2D(32, (3, 3), padding='same', activation='relu')(x)
x = BatchNormalization()(x)
x = Conv2D(32, (3, 3), padding='same', activation='relu')(x)
x = BatchNormalization()(x)

# Финальный слой двумерной свертки, выдающий итоговое изображение
x = Conv2D(shape[-1], (3, 3), activation='sigmoid', padding='same')(x)

model = Model(img_input, x) # указываем модель, с оригинальным изображением на входе в сеть и сжатым-разжатым на выходе из сети
model.compile(optimizer=Adam(lr=0.0001),
              loss='mean_squared_error')

return model # функция вернёт заданную модель
```

Пример

Model: "model_1"

Layer (type)	Output Shape	Param #			
=====					
input_2 (InputLayer)	[(None, 28, 28, 1)]	0	conv2d_13 (Conv2D)	(None, 14, 14, 64)	36928
conv2d_9 (Conv2D)	(None, 28, 28, 32)	320	batch_normalization_15 (Batch Normalization)	(None, 14, 14, 64)	256
batch_normalization_10 (Batch Normalization)	(None, 28, 28, 32)	128	conv2d_14 (Conv2D)	(None, 14, 14, 64)	36928
conv2d_10 (Conv2D)	(None, 28, 28, 32)	9248	batch_normalization_16 (Batch Normalization)	(None, 14, 14, 64)	256
batch_normalization_11 (Batch Normalization)	(None, 28, 28, 32)	128	conv2d_transpose_3 (Conv2D Transpose)	(None, 28, 28, 32)	8224
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 32)	0	batch_normalization_17 (Batch Normalization)	(None, 28, 28, 32)	128
conv2d_11 (Conv2D)	(None, 14, 14, 64)	18496	conv2d_15 (Conv2D)	(None, 28, 28, 32)	9248
batch_normalization_12 (Batch Normalization)	(None, 14, 14, 64)	256	batch_normalization_18 (Batch Normalization)	(None, 28, 28, 32)	128
batch_normalization_13 (Batch Normalization)	(None, 14, 14, 64)	256	conv2d_16 (Conv2D)	(None, 28, 28, 32)	9248
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 64)	0	batch_normalization_19 (Batch Normalization)	(None, 28, 28, 32)	128
conv2d_transpose_2 (Conv2D Transpose)	(None, 14, 14, 64)	16448	conv2d_17 (Conv2D)	(None, 28, 28, 1)	289
batch_normalization_14 (Batch Normalization)	(None, 14, 14, 64)	256	=====		
			Total params: 184,225		
			Trainable params: 183,265		
			Non-trainable params: 960		

Пример

Результаты запуска на MNIST

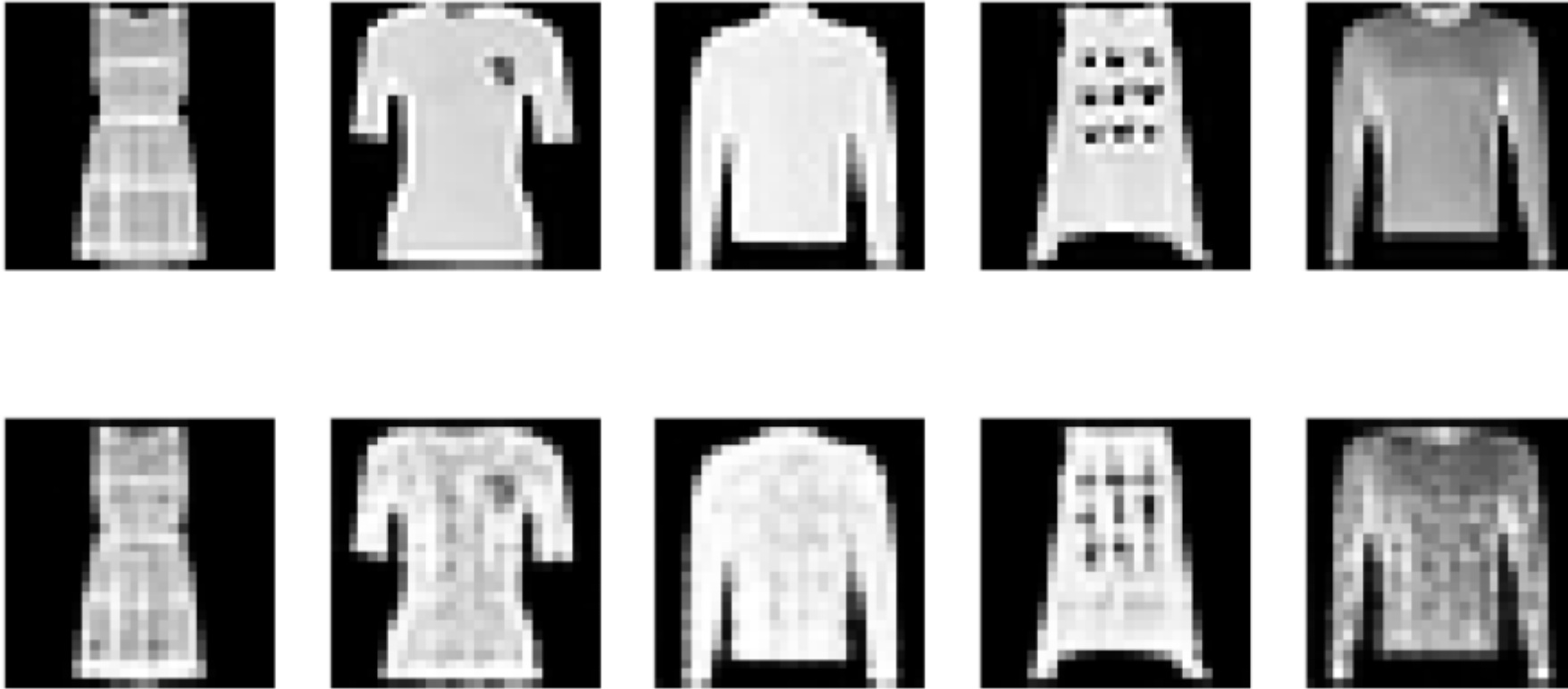
Исходные и восстановленные картинки



Пример

Определение выбросов

1. Загрузка базы Fashion MNIST
2. Приводим к нужной размерности и нормализация
3. Запуск на обученном автокодировщике.
4. Получаем результаты.



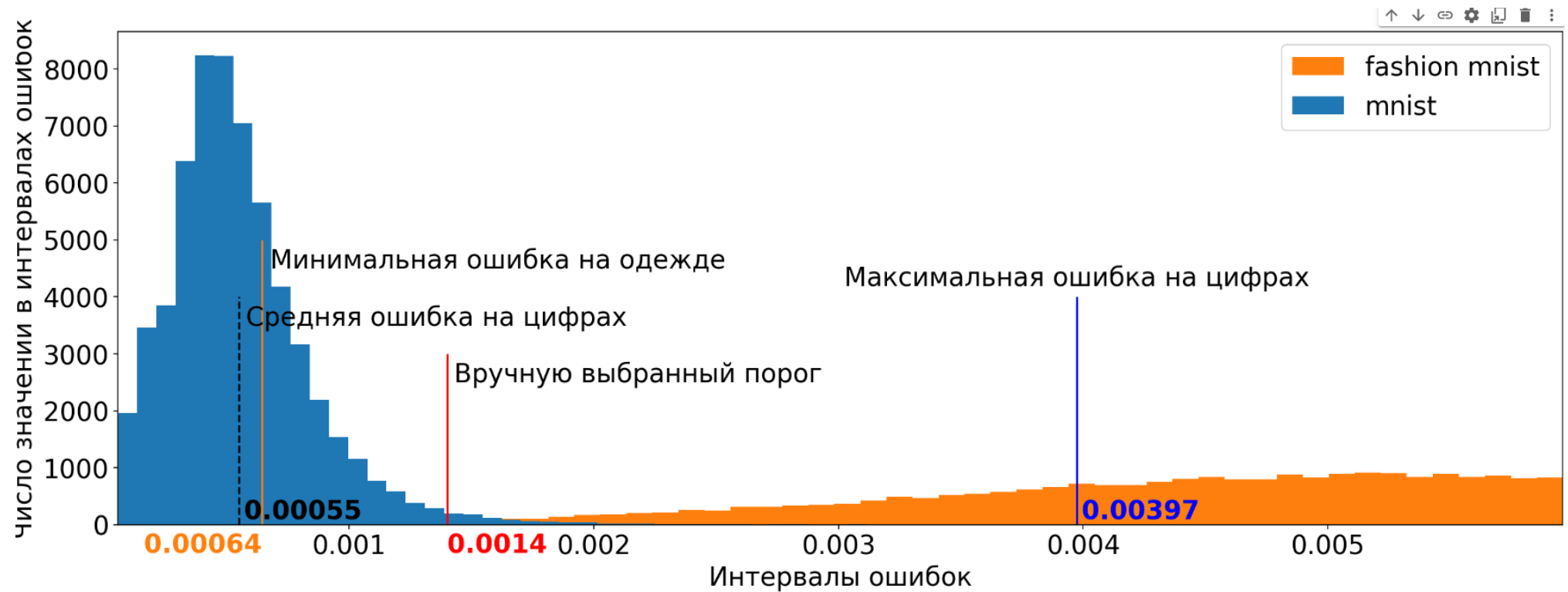
Правильные и восстановленные картинки

Пример

Результаты проверки выбросов на Fashion MNIST по пороговому значению

Средняя ошибка на MNIST: 0.00055
Средняя ошибка на Fashion_MNIST: 0.00749

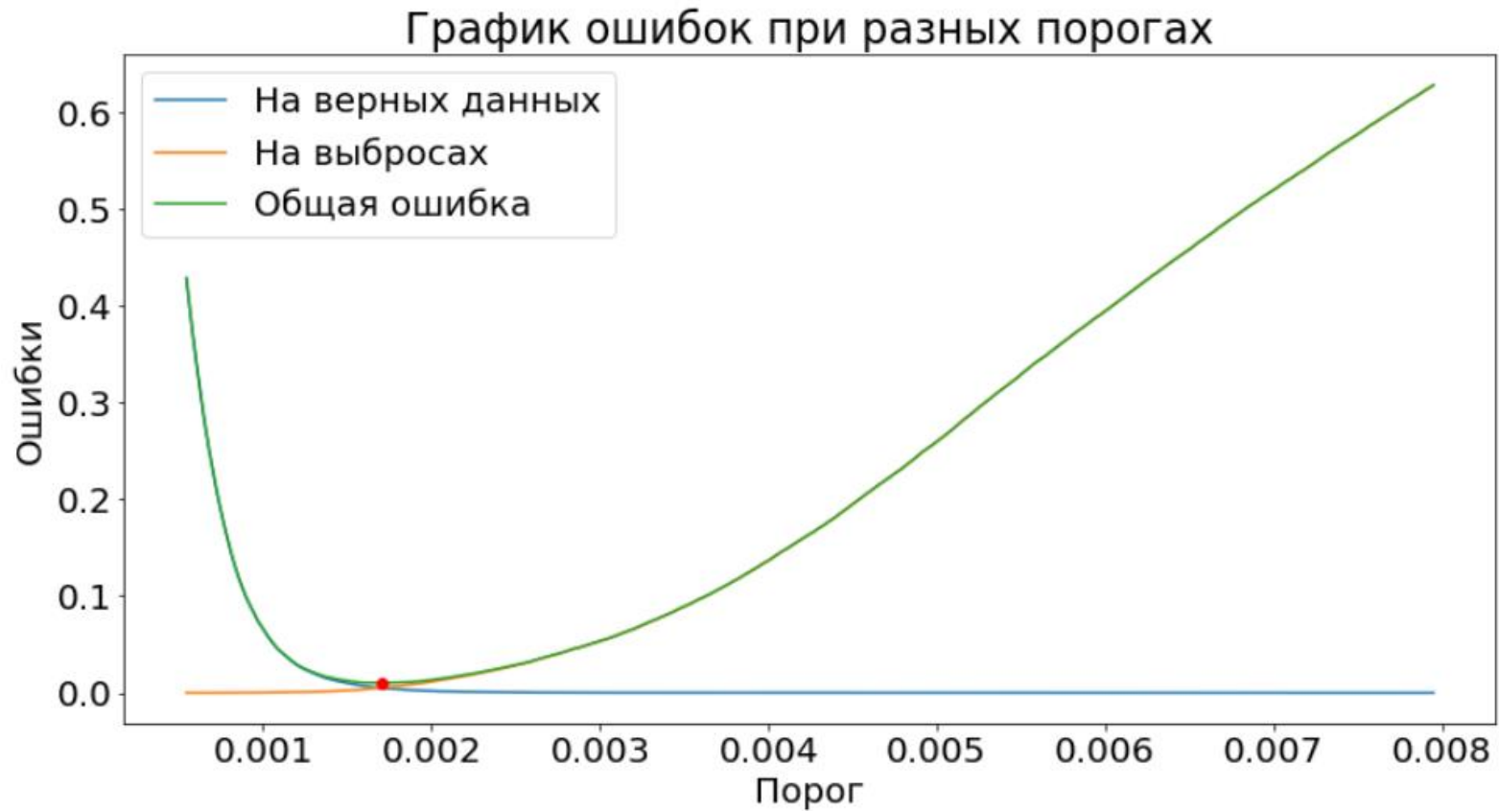
Максимальная ошибка на MNIST: 0.00397
Минимальная ошибка на Fashion_MNIST: 0.00064



Все левее порога считаем данными из mnist, все правее – данными из fashion_mnist (выбросы).

Пример

Результаты проверки выбросов на Fashion MNIST по пороговому значению



Пример 2. Удаление шума

Удаление шума на изображениях

1. Архитектура автокодировщика

1. Энкодер

```
# 3 блока свертки перед сжатием
x = Conv2D(12, 3, padding='same', activation='relu')(inputs)
x = BatchNormalization()(x)

# разрежение (dilation_rate) для выявления мелких деталей изображения
x = Conv2D(24, 3, dilation_rate=2, padding='same', activation='relu')(x)
x = BatchNormalization()(x)

x = Conv2D(48, 3, padding='same', activation='relu')(x)
x = BatchNormalization()(x)

# Сжатие 1
x = Conv2D(96, 3, strides=2, padding='same', activation='relu')(x)
x = BatchNormalization()(x)

x = Conv2D(96, 3, padding='same', activation='relu')(x)
x = BatchNormalization()(x)

# Сжатие 2
x = Conv2D(128, 3, strides=2, padding='same', activation='relu')(x)
x = BatchNormalization()(x)

x = Conv2D(256, 3, padding='same', activation='relu')(x)

# Скрытое пространство
z = BatchNormalization()(x)
```

Пример 2. Удаление шума

Удаление шума на изображениях

1. Архитектура автокодировщика

2. Декодер

```
# Расширение 2
x = Conv2DTranspose(128, 3, strides=2, padding='same', activation='relu')(z)
x = BatchNormalization()(x)

x = Conv2D(128, 3, padding='same', activation='relu')(x)
x = BatchNormalization()(x)

# Расширение 1
x = Conv2DTranspose(96, 3, strides=2, padding='same', activation='relu')(x)
x = BatchNormalization()(x)

x = Conv2D(48, 3, padding='same', activation='relu')(x)
x = BatchNormalization()(x)

x = Conv2D(24, 3, padding='same', activation='relu')(x)
x = BatchNormalization()(x)

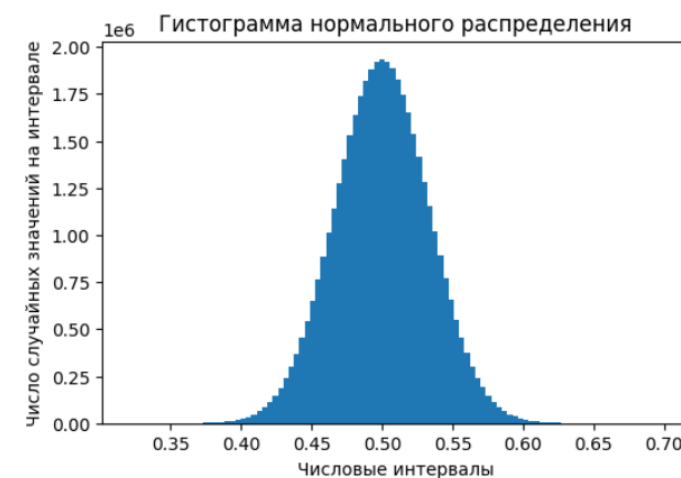
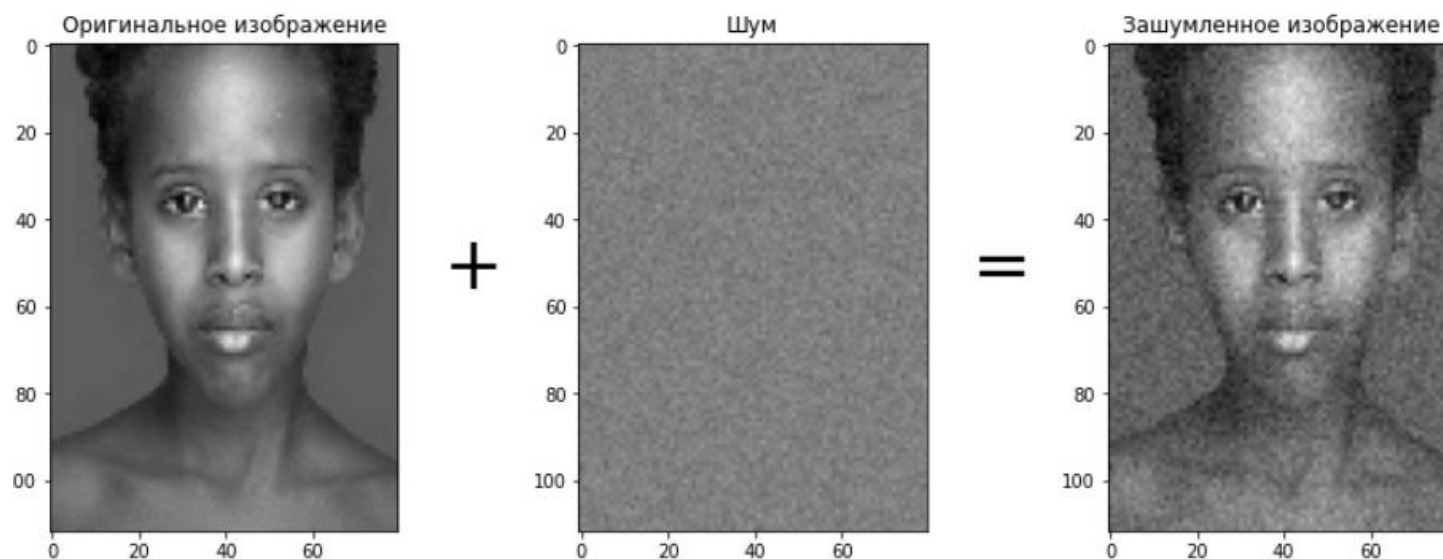
x = Conv2D(8, 3, padding='same', activation='relu')(x)
x = BatchNormalization()(x)

# финальный слой свертки, выход модели
outputs = Conv2D(1, 2, dilation_rate=2, padding='same', activation='sigmoid')(x)

# Сборка модели. На входе оригинальное изображение, на выходе - сжатое-восстановленное
model = Model(inputs, outputs)
```

Пример 2. Удаление шума

2. Создание зашумленного изображения



Пример 2. Удаление шума

3. Создание, обучение и сохранение автокодировщика.

4. Проверка результатов на основе тренировочных данных и вывод изображений.

Оригинал



Зашумлённое изображение



Предсказание



Извлечённый шум

